



Universidad Nacional del Altiplano de Puno

Escuela Profesional de Ingeniería de Sistemas

IEEE Computer Society & IEEE WIE

Rama Estudiantil UNAP

# I Concurso de Programación Competitiva EPIS-UNAP-IEEE 2026

## Problemset

---

8 problemas: A — H

Lenguaje: C++17

### Organizan:

Escuela Profesional de Ingeniería de Sistemas — UNAP

Capítulos Estudiantiles IEEE Computer Society & IEEE WIE

Julio 2026

## Instrucciones generales

- Este cuadernillo contiene **8 problemas**, identificados con las letras **A** a **H**, ordenados de **menor a mayor dificultad** (★ = fácil, ★★ = intermedio, ★★★ = difícil).
- Cada equipo dispone de una única cuenta compartida para enviar soluciones durante todo el concurso.
- Las soluciones se envían y califican a través del sistema DOMjudge. El resultado de cada envío se muestra en la plataforma.
- El puntaje se basa en la cantidad de problemas resueltos; en caso de empate, se usa el tiempo total acumulado (incluyendo penalizaciones por intentos fallidos).
- El código debe compilar en **C++17** (u otro lenguaje habilitado en el sistema) y debe leer desde la entrada estándar y escribir en la salida estándar.
- Ante cualquier duda sobre un enunciado, usar la opción de **clarificaciones (clarifications)** del sistema — no se aceptan preguntas verbales durante el concurso.

| Letra | Problema                        | Límite de tiempo | Dificultad |
|-------|---------------------------------|------------------|------------|
| A     | ABC                             | 1 s              | ★          |
| B     | Simon Dice                      | 1 s              | ★          |
| C     | Puntuación del Concurso ACM     | 1 s              | ★          |
| D     | Baloncesto Uno contra Uno       | 1 s              | ★          |
| E     | Portafolio de Inversión en ETFs | 1 s              | ★★         |
| F     | CD                              | 5 s              | ★★         |
| G     | Inversiones en la Fila          | 2 s              | ★★         |
| H     | Islas en el Mapa                | 2 s              | ★★★        |



Límite de tiempo: 1 segundo(s) | Dificultad: ★

Se te proporcionarán tres números enteros positivos. Se sabe que, al ordenarlos de menor a mayor, corresponden a los valores **A**, **B** y **C**, respectivamente.

Posteriormente se te dará una cadena formada por las letras **A**, **B** y **C**, indicando el orden en el que deben imprimirse dichos números.

Tu tarea es mostrar los números siguiendo exactamente el orden solicitado.

### Entrada

La primera línea contiene tres números enteros positivos. Cada número es menor o igual a **100**. La segunda línea contiene una cadena de tres caracteres formada únicamente por las letras A, B, C, sin espacios.

### Salida

Imprime los tres números en el orden indicado por la cadena, separados por un espacio.

### Restricciones

$$1 \leq \text{valor} \leq 100$$

#### Entrada de ejemplo

1 5 3  
ABC

#### Entrada de ejemplo

6 4 2  
CAB

#### Salida de ejemplo

1 3 5

#### Salida de ejemplo

6 2 4

# B Simon Dice

Límite de tiempo: 1 segundo(s) | Dificultad: ★

En el juego “**Simon Dice**”, un jugador llamado Simón da instrucciones a los demás participantes. La regla principal es muy sencilla: si una instrucción comienza exactamente con la frase “**Simon says**”, todos deben obedecerla; si no, debe ser ignorada.

Tu tarea es escribir un programa que procese todas las instrucciones e imprima únicamente aquellas que deban seguirse.

## Entrada

La primera línea contiene un entero  $N$  ( $1 \leq N \leq 1000$ ), la cantidad de instrucciones. Cada una de las siguientes  $N$  líneas contiene una instrucción de a lo sumo 100 caracteres. La frase “Simon says”, cuando aparece, siempre está al inicio de la línea y va seguida de al menos una palabra.

## Salida

Por cada instrucción que comience exactamente con **Simon says**, imprime el resto de la instrucción (sin incluir esa frase). Las demás instrucciones se ignoran.

### Entrada de ejemplo

```
1
Simon says smile.
```

### Entrada de ejemplo

```
3
Simon says raise your right hand.
Lower your right hand.
Simon says raise your left hand.
```

### Entrada de ejemplo

```
3
Raise your right hand.
Lower your right hand.
Simon says raise your left hand.
```

### Salida de ejemplo

```
smile.
```

### Salida de ejemplo

```
raise your right hand.
raise your left hand.
```

### Salida de ejemplo

```
raise your left hand.
```



# Puntuación del Concurso ACM

Límite de tiempo: 1 segundo(s) | Dificultad: ★

Nuestro nuevo sistema de envíos para concursos de programación mantiene un registro cronológico de todos los envíos realizados por un equipo durante la competencia. Cada registro almacena el minuto del envío, la letra del problema, y el resultado (**right** o **wrong**).

La clasificación se determina por: (1) cantidad de problemas resueltos, y (2) tiempo total, incluyendo una penalización de **20 minutos** por cada envío incorrecto previo sobre un problema que finalmente se resolvió. Si un problema nunca se resuelve, sus intentos incorrectos no generan penalización. Una vez resuelto un problema, cualquier envío posterior sobre ese mismo problema se ignora. Los registros siempre están en orden cronológico no decreciente.

## Entrada

Varias líneas con el formato `m problema resultado`, donde  $1 \leq m \leq 300$ . La entrada termina con una línea que contiene únicamente `-1`.

## Salida

Dos enteros separados por un espacio: la cantidad de problemas resueltos, y el tiempo total (con penalizaciones).

### Entrada de ejemplo

```
3 E right
10 A wrong
30 C wrong
50 B wrong
100 A wrong
200 A right
250 C wrong
300 D right
-1
```

### Entrada de ejemplo

```
7 H right
15 B wrong
30 E wrong
35 E right
80 B wrong
80 B right
100 D wrong
100 C wrong
300 C right
300 D wrong
-1
```

### Salida de ejemplo

```
3 543
```

### Salida de ejemplo

```
4 502
```

# D Baloncesto Uno contra Uno

Límite de tiempo: 1 segundo(s) | Dificultad: ★

Alicia y Bárbara jugaron partidos amistosos de baloncesto uno contra uno. Cada lanzamiento exitoso otorga 1 o 2 puntos. El primer jugador que alcance 11 puntos gana, con una excepción: si ambos llegan a 10–10, a partir de ahí se gana solo con una ventaja de 2 puntos.

Cada anotación se registró como una letra (A o B) seguida de un número (1 o 2), en orden cronológico. Reconstruye quién ganó.

## Entrada

Una única línea con el registro completo (máximo 200 caracteres), formada por pares letra+número sin espacios. Se garantiza que corresponde a un partido válido y finalizado.

## Salida

Un único carácter: A si ganó Alicia, B si ganó Bárbara.

### Entrada de ejemplo

A2B1A2B2A1A2A2A2

### Entrada de ejemplo

A2B2A1B2A2B1A2B2A1B2A1A1B1A1A2

### Salida de ejemplo

A

### Salida de ejemplo

A



# Portafolio de Inversión en ETFs

Límite de tiempo: 1 segundo(s) | Dificultad: ★★

Cuentas con un capital exacto  $C$  que deseas invertir íntegramente. Tienes a tu disposición una lista de  $N$  ETFs con precios enteros únicos. No puedes comprar fracciones ni repetir un ETF. Determina cuántas combinaciones distintas de ETFs suman exactamente tu capital.

## Entrada

La primera línea contiene el capital  $C$  ( $1 \leq C \leq 10^4$ ) y la cantidad de ETFs  $N$  ( $1 \leq N \leq 20$ ). La segunda línea contiene  $N$  enteros separados por espacios, los precios de cada ETF ( $1 \leq P_i \leq 10^4$ ).

## Salida

Un único entero: la cantidad de combinaciones válidas que suman exactamente el capital  $C$ .

### Entrada de ejemplo

```
100 5
20 30 40 50 60
```

### Salida de ejemplo

```
2
```



Límite de tiempo: 5 segundo(s) | Dificultad: ★★

Jack y Jill venden una copia de cada CD que ambos poseen. Ninguno tiene copias repetidas dentro de su propia colección, y los números de catálogo están ordenados de forma creciente.

### Entrada

Varios casos de prueba. Cada uno inicia con dos enteros **N** y **M** ( $1 \leq N, M \leq 10^6$ ), seguidos de **N** líneas con los catálogos de Jack y **M** líneas con los de Jill (cada catálogo  $\leq 10^9$ ). Termina con una línea **0 0**, que no se procesa. La suma total de **N** y **M** no supera  $6 \times 10^6$ .

### Salida

Para cada caso, un entero: la cantidad de CD que tienen en común.

#### Entrada de ejemplo

```
3 3
1
2
3
1
2
4
1 2
10
5
6
0 0
```

#### Salida de ejemplo

```
2
0
```





# Inversiones en la Fila

Límite de tiempo: 2 segundo(s) | Dificultad: ★★

Un grupo de  $n$  estudiantes está formado en una fila para la foto anual. Cada estudiante tiene una estatura. Un profesor define una **inversión** como un par de posiciones  $(i, j)$  con  $i < j$  tales que el estudiante en la posición  $i$  es **más alto** que el estudiante en la posición  $j$  (un par “desordenado”: el más alto va antes que el más bajo). Cuenta cuántas inversiones hay en la fila.

## Entrada

La primera línea contiene un entero  $n$  ( $1 \leq n \leq 200000$ ), la cantidad de estudiantes. La segunda línea contiene  $n$  enteros  $a_1, \dots, a_n$  ( $1 \leq a_i \leq 10^9$ ), las estaturas en el orden en que están parados.

## Salida

Un único entero: la cantidad de inversiones en la fila. El resultado puede ser grande (hasta cerca de  $2 \times 10^{10}$ ), imprimir con un tipo de 64 bits.

## Nota para los concursantes

$n$  hasta 200 000 exige un algoritmo  $O(n \log n)$  (por ejemplo, conteo de inversiones con *merge sort*, o un Fenwick/BIT con coordenadas comprimidas). Un enfoque  $O(n^2)$  dará *Time Limit Exceeded* en los casos grandes. Los valores repetidos no cuentan como inversión.

### Entrada de ejemplo

```
6
5 2 6 1 3 4
```

### Salida de ejemplo

```
8
```

# H Islas en el Mapa

Límite de tiempo: 2 segundo(s) | Dificultad: ★★★

Se te da un mapa representado como una grilla de  $R$  filas por  $C$  columnas, donde cada celda contiene un carácter: 1 (tierra) o 0 (agua). Una isla se define como un grupo de celdas de tierra conectadas entre sí horizontal o verticalmente (arriba, abajo, izquierda, derecha). Las celdas conectadas solo en diagonal **no** se consideran parte de la misma isla. Determina cuántas islas hay en el mapa.

## Entrada

La primera línea contiene dos enteros  $R$  y  $C$  ( $1 \leq R, C \leq 1000$ ), las dimensiones del mapa. Las siguientes  $R$  líneas contienen  $C$  caracteres cada una (0 o 1, sin espacios).

## Salida

Un entero: la cantidad de islas en el mapa.

### Entrada de ejemplo

```
4 5
11110
11010
11000
00000
```

### Salida de ejemplo

```
1
```

### Entrada de ejemplo

```
4 5
11000
11000
00100
00011
```

### Salida de ejemplo

```
3
```